

Daily Scholar Papers Report — 2026-09-08

2026-09-08

Daily Scholar Papers Report — 2026-09-08

[Download PDF](#)

Window covered: 2026-09-07 → 2026-09-08 (Google Scholar alerts + user-curated self-emails, last 24 h). Three alert threads arrived, none previously consumed; the standing 7-day liveness sweep returned seventeen threads, fourteen of them already accounted for by earlier runs. No backlog, and no self-forwarded papers this window.

Executive Summary

Eight candidates, five read, and the thread running through them is **what you are allowed to stop doing** — which measurements you can truncate, which artefacts you can stop hand-building, which representations you can stop paying for.

The most quantitative answer comes from **EarlyEval**, which asks why an agentic benchmark run must finish. A single pass of a frontier model over SWE-bench Verified costs **\$715–\$935**, and SWE-bench Multimodal reaches **\$2,270** — paid again after every prompt tweak. The paper’s observation is that the outcome is usually legible long before the run ends, so it trains a pair of cheap gradient-boosted classifiers over trajectory prefixes and halts at the first confident crossing: **26.0 % of steps, 32.7 % of input tokens and 28.7 % of output tokens eliminated at a 1.1-point resolve-rate distortion**, with the leaderboard order essentially intact ($p = 0.991$, 81 % of agents unmoved). But the finding worth carrying away is an accident of the ablation table: dual step reduction *equals the sum* of its two halves ($-10.6 \% + -15.4 \% = -26.0 \%$), which — since a run halts at the first crossing — means the success and failure predictors almost never fire on the same trajectory. Positive and negative evidence do not co-occur inside a run.

Towards Autonomous Software Engineering asks the same question at the scale of a discipline, and answers it with vocabulary rather than numbers. Its three levels — code autonomy, pipeline autonomy, demand autonomy — are organised around one invariant: *preserving and faithfully executing human intent as direct control recedes*. The sharpest claim is structural, not predictive: at Level II an agent writes both the implementation and the tests, so artefact-level validation stops being evidence, and assurance has to move from the artefacts to the agents that produce them — their specifications, skills, tool permissions, memory and traces. The paper also does the thing position papers usually skip, arguing against itself in a dedicated section.

Then the paper that tries to stop hand-building datasets and reports honestly that it mostly cannot. **LLM-based vulnerability injection** into Solidity generates nearly **1,000 candidate variants** and finishes with **32 confirmed contracts across 25 vulnerability types — a 16.58 % survival rate**. The negative result is the contribution, because it is decomposed: the collapse is concentrated in one step, **business-logic verification, which cuts survivors from 89 to 44**. Syntactic injection is easy; not breaking the contract

while doing it is the hard part. The authors then publish their own selection bias — the surviving ground truth skews to locally-checkable defects, **11 of the 25 types are** `Checking-class` — and warn readers not to over-read tool comparisons built on it. A separate, cleaner finding falls out along the way: **13 of 32 validated contracts (40.6 %) could not be analysed by any of the three static analysers**, and the same failures occur on the *unmodified* originals, so the cause is legacy Solidity, not the injection.

The last full read is a small study whose detection numbers are unremarkable and whose *budget* numbers are not. On the question of how to fit a structured code representation into a lightweight local model, **behaviour trees cover 260 of 260 longer Java samples at a 4096-token window while ASTs cover 86 — and even at 16 384 tokens ASTs still reach only 222**. That is a clean statement about representation cost under a fixed context budget, and it survives independently of whether the accompanying F1 scores mean anything.

Five papers, one instinct: find the point past which the extra work stops buying evidence.

Outstanding: 2 · Keep: 2 · Borderline High-Priority: 1

A note on depth and on links. Four of the five papers were read in full text from the publisher or author PDF; every number below is transcribed from the source. **None of the five carries a Creative Commons, ACM or IEEE licence notice**, so no PDF has been mirrored into this repository and no figures are reproduced — each paper carries a canonical arXiv, publisher or author-hosted link instead. One paper (ConFluxFuzz) sits behind IEEE Xplore and is not yet indexed in any open source, so it appears as a short pointer rather than a summary; it is listed rather than dropped so that a reader who has institutional access can follow it up.

Highlighted Papers

Title	Authors	Venue	Link
EarlyEval: Cheaper Agent Evaluation via Early Outcome Prediction	Y. Shi, Z. Sun, J. Dong, C. Wan, D. Lo, X. Gu	arXiv preprint, cs.CL, 2 Sep 2026 (SJTU · SMU · ECNU · Shanghai Innovation Institute)	arXiv:2609.02783
Towards Autonomous Software Engineering	H. Wang, R. Meng, Z. Ye, A. Gu, N. Jain, X. Liu, S. Chaudhuri, T. Zimmermann, S. Gulwani, A. Solar-Lezama, I. Stoica, D. Song	Position paper, 2026 (UC Berkeley · CISPAC · MIT · Cursor · UT Austin · UC Irvine · Microsoft)	author PDF · Scholar lookup
Automated Vulnerability Injection in Smart Contracts Using Large Language Models	L. Migliaccio, R. Natella, N. Ivaki, N. Laranjeiro, M. Vieira	arXiv preprint, cs.SE, 2 Sep 2026 (Napoli Federico II · GSSI · Coimbra · UNC Charlotte)	arXiv:2609.02624
Towards Behavior Tree-Guided Vulnerability Detection with Lightweight LLMs	E. Basic, A. Giaretta	arXiv preprint, cs.CR, 1 Sep 2026 (Örebro University)	arXiv:2609.01758
ConFluxFuzz: Control-Flow Restructuring and Freshness-Decayed		IEEE Transactions, 2026	IEEE Xplore 11674189

Title	Authors	Venue	Link
Scheduling for Processor Fuzzing	R. Fang, H. Wang, X. Chen, W. Cui, N. Cui, L. Chen, G. Shi, et al.		

Papers

1.1 · AGENT EVALUATION · A benchmark run does not have to finish — 26 % of steps and 33 % of input tokens removed at a 1.1-point resolve-rate cost 

EarlyEval: Cheaper Agent Evaluation via Early Outcome Prediction

Authors: Yuling Shi (Shanghai Jiao Tong University), Zhensu Sun (Singapore Management University), Junsen Dong (Shanghai Jiao Tong University), Chengcheng Wan (East China Normal University · Shanghai Innovation Institute), David Lo (Singapore Management University), Xiaodong Gu (Shanghai Jiao Tong University). First two authors contributed equally.

Venue: arXiv:2609.02783v1, cs.CL, 2 Sep 2026. No conference or journal is named in the paper.

Links. [arXiv:2609.02783](https://arxiv.org/abs/2609.02783). **Licence: none stated** — no Creative Commons, ACM or IEEE notice appears in the PDF, so nothing is mirrored here and no figures are reproduced. **Evidence base: full text retrieved and read. Every number below is transcribed from the paper.**

The gap being targeted

Agentic benchmarks are expensive in a way that compounds. Each task is a multi-step rollout — reading files, running commands, calling tools, retrying — so every task issues dozens of model calls, and the bill is paid again on every iteration of a development cycle. The paper grounds this in the OpenHands Index, a public leaderboard recording measured dollar cost, with figures retrieved June 2026:

Benchmark	#Tasks	Claude 5	GPT-5.5	Gemini 3.1 Pro
SWE-bench Verified	500	\$715	\$760	\$935
SWT-bench	500	\$735	\$460	\$810
Commit0	54	\$674	\$300	\$64
GAIA	165	\$1,305	\$122	\$297
SWE-bench Multimodal	517	\$2,270	\$1,453	\$641

Every figure is one pass of one agent configuration. A team that re-evaluates after each prompt, scaffold or model change, across every baseline, multiplies it by dozens.

The existing answer is benchmark distillation — anchor tasks, proxy subsets — which shrinks the *number* of tasks and leaves the cost of each retained task exactly where it was. EarlyEval attacks the orthogonal axis, and the two compose rather than compete.

The move: halt the run, not the benchmark

The observation is that outcomes are frequently foreseeable. The paper’s worked example is a released OpenHands trajectory on `tianocore/edk2-pytool-library` that runs 45 steps: the agent reproduces the bug by step 20, makes its single source modification at step 23, and then never touches the source again while continuing to test in various directions. An observer with the gold patch could have called it at step 23, at roughly half the cost.

Formally (§III-A), an agent on task t produces a trajectory $\tau = (e_1, e_2, \dots, e_T)$ where each event e_k records an action and its resulting observation; at termination the benchmark assigns a binary score $y \in \{0, 1\}$. An early-outcome predictor monitors the run and may at any step $k < T$ issue a prediction $\hat{y}$ and halt, recording $\hat{y}$ in place of y ; otherwise it lets the agent continue.

Two classifiers, not one. Every historical trajectory is expanded into all its prefixes $\tau:k = (e_1, \dots, e_k)$, each paired with the trajectory’s *final* label. A success predictor h^+ is trained against $y = 1$ and a failure predictor h^- against $1 - y = 1$. Training them separately lets positive and negative evidence accumulate independently — the paper’s stated rationale is that success and failure are signalled by fundamentally asymmetric behaviours — and it creates an explicit *unconfident region* where both outputs are low and the agent is simply allowed to keep going. Trajectories shorter than 10 steps are discarded. Each prefix is weighted by $1/(T+1)$ so that long trajectories cannot dominate the loss.

Features (Table II), 517 dimensions in three families. *Behavioral* (115): activity counts, last-step properties, event timing, working-pattern signals such as repeated-action and consecutive-read streaks, and error/test status indicators. *Textual* (320): TF-IDF over word unigrams and bigrams for the task prompt, action history and environment feedback, each block vectorised independently and compressed by truncated SVD to 64 or 128 dimensions to keep per-step inference cheap. *Reference* (82, optional): gold-patch descriptors plus Jaccard overlap between the files, API symbols and test names the agent has touched and those in the reference solution.

Calibration and halting (§III-E). Tree ensembles distort probability scales, so raw scores are recalibrated by Platt scaling — a one-dimensional logistic regression

$$p = \sigma(a \cdot \text{logit}(\hat{s}) + b)$$

fitted on a held-out validation split under the same per-prefix weights. The transformation is monotonic, so it preserves ranking and AUC; its only job is to make thresholds mean the same thing across both predictors and all folds. The run halts at the first step where $p^+ \geq s$ or $p^- \geq f$; ties break chronologically.

The backbone is LightGBM, chosen because a several-hundred-dimensional vector scores in well under a millisecond on one CPU core — an LLM judge invoked at every step would eat the compute the framework is trying to save.

Evaluation

Three benchmarks, more than 21,000 outcome-labelled trajectories: **SWE-bench Verified** (500 issues, one scaffold $\times$ 16 base LLMs, 7,805 trajectories), **TerminalBench** (89 shell tasks, 37 scaffold+LLM configurations, 6,757 trajectories), **Toolathlon** (108 tool-use tasks, 22 base LLMs $\times$ 3 rollouts, 7,116 trajectories). Neither TerminalBench nor Toolathlon releases per-task reference solutions, so the reference feature family is disabled on both.

The protocol is **leave-one-agent-out**: one agent held out, predictors trained only on the rest. TerminalBench gets two additional leakage controls, since a held-out agent’s model or scaffold may individually appear in training — one split removes every trajectory sharing the held-out model, the other every trajectory sharing the scaffold.

$\Delta\text{Steps} = \Sigma S_{\text{early}} / \Sigma S_{\text{full}} - 1$, so a negative value is the fraction of steps eliminated.

Results

At the recommended operating points:

Benchmark	Threshold	ΔSteps	$\Delta\text{Token}_{\text{in}}$	$\Delta\text{Token}_{\text{out}}$	$\Delta \text{Pass}@1 $
SWE-bench Verified	0.95	−26.0 %	−32.7 %	−28.7 %	1.1 pp
TerminalBench (no same model)	0.90	−25.4 %	−42.7 %	−27.9 %	2.1 pp
TerminalBench (no same scaffold)	0.85	−17.7 %	−29.2 %	−17.4 %	2.0 pp
Toolathlon	0.90	−23.0 %	−44.1 %	−29.4 %	0.9 pp

Token savings exceed step savings throughout, because halting also removes the compounding cost of an expanding context window. The threshold is a clean dial: dropping SWE-bench from 0.95 to 0.75 takes step reduction from 26.0 % to **63.4 %** while distortion rises from 1.1 to 4.1 points.

Leaderboards survive. Spearman’s ρ is **0.991** on SWE-bench Verified, **0.994** on Toolathlon and on the no-same-scaffold split, **0.959** on the harder no-same-model split; between 59 % and 81 % of agents keep their exact ordinal position, and on SWE-bench only three adjacently ranked agents move by a single rank.

Three results are more interesting than the headline.

The additivity. At nearly every operating point, dual step reduction equals the sum of the success-only and failure-only reductions — $-10.6 \% + (-15.4 \%) = -26.0 \%$ on SWE-bench at 0.95. Because a run halts at the *first* crossing, exact additivity implies the two predictors almost never fire on the same trajectory. Positive and negative evidence rarely coincide within a run, which is a claim about the structure of agent trajectories rather than about the tool.

The asymmetry, which the authors state plainly. The success predictor is reliable only on SWE-bench Verified (88.3–93.9 % precision across thresholds). On TerminalBench’s no-same-scaffold split it falls to **61.4–69.0 %**, and on Toolathlon its coverage collapses to zero at thresholds ≥ 0.90 — the entire saving on those two benchmarks comes from the failure predictor, which stays robust everywhere (96.7 % on SWE-bench, 89.4–96.6 % on TerminalBench, 96.6–99.4 % on Toolathlon). The offered mechanism is specific and testable: a scaffold dictates the structural rhythm of a trajectory, so an unseen scaffold perturbs exactly the behavioural features the predictor leans on, whereas an unseen base model leaves that skeleton comparatively stable. Peak success precision drops from 82.7 % (no same model) to 69.0 % (no same scaffold), and attainable step reduction from 25.4 % to 17.7 %.

The reference ablation, which is the reusability result. Removing the entire reference-solution family moves coverage only 34.8 % → 32.1 % and step savings 26.0 % → 24.7 %. Removing the behavioural family is far more damaging (coverage 23.4 %, savings 16.4 %). Within any family, no single subgroup matters — ablating one behavioural group shifts coverage by at most 0.5 points. Signal is redundantly encoded, which is what lets the method work on benchmarks that release no gold patches. Against alternative backbones at the same threshold, LightGBM (34.8 % coverage, 95.0 % accuracy) beats a direct MLP (26.9 %, 87.9 %), both logistic-regression variants, and a LoRA-fine-tuned Qwen-0.5B judge (18.7 %, 90.7 %).

Why it is worth the time

The framing is the contribution: *early outcome prediction* is a second axis of evaluation efficiency that composes with distillation instead of replacing it, and the mechanism is cheap enough that it can sit inside any harness without materially changing the bill. The additivity observation and the scaffold-versus-model asymmetry are both reusable findings about agent trajectories independent of this particular tool. And the honesty is load-bearing — a paper that tells you its success head collapses to zero coverage on one of its three benchmarks is a paper whose other numbers are easier to trust.

1.2 · POSITION PAPER · Three levels of SE autonomy, one invariant — and at Level II the agent writes both the code and the tests, so artefact validation stops counting as evidence 🍌 😊 🍌

Towards Autonomous Software Engineering

Authors: Hao Wang, Zhe Ye, Xiaoyuan Liu, Ion Stoica, Dawn Song (UC Berkeley); Ruijie Meng (CISPA); Alex Gu, Armando Solar-Lezama (MIT); Naman Jain (Cursor); Swarat Chaudhuri (UT Austin); Thomas Zimmermann (UC Irvine); Sumit Gulwani (Microsoft).

Venue: Position paper, 2026. Author-hosted PDF; no conference, journal or arXiv identifier appears in the document.

Links. [author PDF](#) · [Scholar lookup](#). **Licence: none stated** — no Creative Commons or publisher notice appears in the PDF, so nothing is mirrored here and no figures are reproduced. **Evidence base: full text retrieved and read, including the appendices.**

The gap being targeted

A large literature discusses software engineering in the AI era, and almost all of it studies *task-level* autonomy inside a human-led workflow — AI-assisted coding here, automated program repair there — with the implicit assumption that a human stays in the lifecycle. The paper's claim is that this framing has already been overtaken: responsibility is migrating stage by stage, and without a shared vocabulary for that migration, progress is ad hoc and the risks are invisible. So the proposal is to study software engineering **as a trajectory of substitutions for increasing autonomy**, and to be explicit about what shifts at each step.

The taxonomy

Anchored in the classical lifecycle — requirement curation, system design, implementation, testing, deployment — and modelled on the levels of driving automation:

Level I — Code Autonomy. AI takes full ownership of system design and implementation without line-by-line human approval. Humans retain the oversight-critical stages: testing, deployment, and demand

proposal. The operative difference from present practice is that review happens at pull-request granularity rather than through co-authoring.

Level II — Pipeline Autonomy. AI owns design through testing and deployment. Humans neither author nor review code; their role narrows to specifying high-level demands, accepting the resulting behaviour, and repeating. This level *presumes* a sufficiently complete specification and comprehensive automated verification, because nobody inspects the intermediate artefacts.

Level III — Demand Autonomy. AI also decides what to build, identifying demands from operational signals, user behaviour and security advisories rather than receiving them. Autonomy remains bounded by a founding mission defined by humans at inception; reconciling emergent demands with that original intent is itself named as a core open challenge.

Across all three the authors identify one unifying concern — **preserving and faithfully executing human intent as direct human control recedes** — and organise everything else around it.

Three orthogonal dimensions cut across the levels, which is what stops the taxonomy from being a simple ladder. *Specification granularity*: weak specifications push even low-level systems into high-autonomy difficulties, strong ones do the reverse. *Temporal autonomy*: ticket-level → sprint-level → release-level → continuous operation, independent of level, so a Level I system can run at sprint scale while a Level II system stays stuck at ticket scale on context-window limits. *Oversight mode*: collaborative co-specification, action-level approval, plan-level approval, boundary/guardrail autonomy, monitor-only with rollback.

The authors are careful about two things. They do not present the trajectory as inevitable — the argument is that stage-level responsibility transfer is *already happening* and deserves vocabulary regardless of where it terminates. And they explicitly warn against level-skipping: they observe Level II-style pipelines already deployed where outputs cannot be reliably verified, and argue for **explicit level gating** on technical *and* societal criteria, noting that high-assurance domains such as medical devices and avionics are likely to plateau at Level I indefinitely, gated by certification regimes that presuppose human authorship and liability.

The structural shifts

Specifications become the primary artefact. At Level II, systems operate from specifications nobody verifies end-to-end, which exposes two problems at once: natural-language demands are ambiguous and underspecified, and many quality dimensions — maintainability, UI/UX suitability, refactoring correctness — have no gold oracle at all. The authors anticipate **specification distillation**: rather than authoring a requirements document up front, developers converse with agents and the agent compiles the accumulated dialogue into a durable specification. They also push back on a naive reading of Level III — specification synthesis is not an agent silently inventing intent, but a negotiation with stakeholders, customers and regulators.

Software takes new forms. Neurosymbolic (deterministic code on the paved paths, neural components on the novel ones), self-evolving (the deployed system as a continuously edited artefact, which dissolves the release boundary and creates an auditing problem — you must pin an exact checkpoint while the system keeps changing), conversational (natural language replacing fixed UIs, so the boundary between *using* and *modifying* software blurs), and ephemeral (generated on demand, discarded after, which sheds lifecycle concerns but not intent-alignment ones).

Testing has to move from artefacts to agents. This is the sharpest argument in the paper. AI-generated code is structurally diverse — the same specification yields substantially divergent implementations — which already degrades tooling that relies on recurring patterns and predictable control flow. Worse, at higher autonomy the burden of writing and validating tests shifts to AI, and **an agent that writes both the implementation and its tests can be perfectly consistent and still wrong**. Delegating the check to a separate verifier agent does not fix it: verifier and generator can talk past each other, or co-adapt until the tests merely ratify the implementation's bugs. The conclusion is that assurance must extend from artefacts to the agents producing them — auditing specifications, skills, tool permissions, memory and traces as first-class objects.

Coordination stops mirroring human org charts. Today's multi-agent orchestration copies human hierarchies, which exist because of human limits on bandwidth, attention and span of control. Agents can fork and merge state, broadcast context, and run consensus protocols that are impractical for people, so the authors predict AI-native coordination structures — alongside new failure modes they name directly, including slopsquatting and toxic-skill propagation.

Organisations compress. Higher autonomy means smaller teams focused on specification, security, governance and accountability. The authors argue this is a more fundamental compression than high-level languages or cloud infrastructure produced, because entire categories of labour are absorbed rather than accelerated — and they anticipate new organisational functions and new kinds of firm (third-party AI code auditors, certification bodies for AI-generated software).

The research agenda, and where it is concrete

The strongest section is the one on specification and verification, because it names three specific obstacles rather than gesturing at difficulty. Repository-scale specification benchmarks already outpace what frontier models can produce, so function-level results overstate readiness. Recent analysis argues any auto-formalisation of natural-language intent must ultimately defer to a human to confirm the specification matches intent — which raises the question of what a genuine Level III pipeline could even look like. And almost no published method addresses specification drift across many development cycles, with the benchmarks that do probe it reporting sharp degradation over successive iterations. The authors' call is correspondingly concrete: repository-scale specification-synthesis datasets, and benchmarks that measure specification-implementation consistency and drift across cycles.

On security and governance they anchor to things that exist: OWASP's Top 10 for Agentic Applications, Microsoft's Agent Governance Toolkit and AWS's Agentic AI Security Scoping Matrix as first-pass policy primitives; EU AI Act high-risk obligations from August 2026 and the Colorado AI Act from June 2026 as forcing functions. They identify two limits on automatic auditing — current systems inherit their underlying analysers' rule sets, so structurally novel AI-generated patterns evade them; and defences against adaptive prompt injection remain weak. Their position on liability is direct: if an agent-built application leaks customer data, the accountability sits with the organisation that deployed the agent, not the model provider — and that is itself a strong incentive to retain human oversight.

Where it argues against itself

The paper devotes a full section to two alternatives, and this is the part that most repays reading.

The durable human-AI partnership view. Some activities may be categorically non-delegable regardless of capability: bearing legal or organisational accountability requires not just competence but standing to bear consequences, and curating demands can rest on tacit knowledge, negotiation and institutional

legitimacy that exist nowhere in text. The authors concede this is a real structural difficulty and an open problem. Their reply is twofold — partnership and autonomy are not exclusive, since oversight mode cuts across all levels, so a system can run autonomously while an organisation retains legal responsibility; and the partnership view should be read as a falsifiable prediction about *which classes of software stay human-governed* rather than an abstract endpoint.

The scaling view. The bottlenecks are artefacts of current training regimes and will dissolve as models improve. The authors agree models will keep improving, but argue formal guarantees remain necessary for ultra-low-error-tolerance domains, that scaling provides no guarantees by itself, and — the more interesting half — that scaling cannot resolve challenges arising from the *collaborative* nature of software engineering: specification ambiguity, human–AI collaboration, long-horizon maintenance. Those sit outside the pure capability axis and are precisely what recent progress on implementation benchmarks has left untouched.

Why it is worth the time

Position papers earn their place by supplying vocabulary that survives the specific predictions, and this one does. The level taxonomy plus three orthogonal dimensions is immediately usable for situating a piece of work — “this is Level I at sprint-level temporal autonomy under plan-level approval” is a more informative description than most abstracts manage. The argument that artefact-level validation stops being evidence once one agent authors both code and tests is a clean statement of why verification research does not simply scale with model capability. And the willingness to pre-register and answer its own falsifiers is a standard more position papers should be held to. Read it for the taxonomy and Section 5; the predictions are explicitly labelled speculative by the authors and can be taken or left.

1.3 · DATASET CONSTRUCTION · Nearly 1,000 LLM-injected Solidity variants yield 32 confirmed vulnerable contracts — a 16.58 % survival rate, and the bottleneck is semantics, not syntax 

Automated Vulnerability Injection in Smart Contracts Using Large Language Models

Authors: Luca Migliaccio (Università degli Studi di Napoli Federico II), Roberto Natella (Gran Sasso Science Institute), Naghmeh Ivaki (University of Coimbra, CISUC/LASI), Nuno Laranjeiro (University of Coimbra, CISUC/LASI), Marco Vieira (University of North Carolina at Charlotte).

Venue: arXiv:2609.02624v1, cs.SE, 2 Sep 2026. IEEE two-column formatting, but no conference or journal is named in the paper.

Links. [arXiv:2609.02624](https://arxiv.org/abs/2609.02624). A replication package with all code, data and scripts is cited by the authors.

Licence: none stated — no Creative Commons or IEEE copyright notice appears in the PDF, so nothing is mirrored here and no figures are reproduced. **Evidence base: full text retrieved and read. Every number below is transcribed from the paper.**

The gap being targeted

Judging a smart-contract vulnerability detector requires contracts whose vulnerabilities are known. Such datasets are scarce, small, manually curated, narrow in type coverage, and — the decisive practical problem — hard to extend, because adding a type or a contract means repeating the manual effort. That bottleneck bites hardest on AI-based detectors, which need volume.

Injection is the standard escape: seed vulnerabilities into safe contracts so ground truth exists by construction, the way fault injection supports dependability evaluation. Prior smart-contract work covers

a narrow slice — SolidiFI seeds seven types from predefined patterns, MuSe mutates six types, a GAN-based approach inserts fragments covering seven — and, as the paper’s own comparison table records, none of them verifies that the injected contract still does what it originally did.

The move: assess, inject, then disbelieve the result four times

Three phases. **Phase I** selects targets: 2,862 contracts from SmartBugs filtered to those with no known vulnerabilities, then narrowed on code size, control-flow complexity and external interactions to 14 diverse contracts. **Phase II** uses one LLM to assess which of the 49 OpenSCV vulnerability types can plausibly be injected into each contract, a second to perform an in-place injection under explicit constraints, and a deduplication step. The models are small and locally run — Qwen2.5-Coder 14B (Q4) for assessment, Meta-Llama-3 8B (Q8) for injection, on Kaggle P100 hardware — which matters for the diversity finding below.

Phase III is the paper: four sequential filters, each of which is allowed to kill a candidate. *Step A* compilation and execution. *Step B* injection verification — was the intended vulnerability actually introduced, and were the constraints respected. *Step C* business-logic verification — does the contract still behave as it did apart from the vulnerability. *Step D* vulnerability confirmation — is the defect present, meaningful and exploitable, judged against the expected adversarial effect from each type’s OpenSCV specification.

Contrast with SolidiFI is instructive and the paper draws it carefully. SolidiFI appends self-contained functions and state variables alongside the contract, leaving original code paths untouched; on the shared `tx.origin` case it adds a standalone function checking a parameter while the contract’s own access-control modifier stays intact. This work rewrites that modifier in place. The authors note the two methods pursue different goals — seeding detectable patterns versus producing realistic in-place variants — so survival rates are not directly comparable.

Results

RQ1 — assessment. Manual validation of 108 assessment decisions: **75.9 % accuracy, 95.5 % precision, 73.6 % recall**, with only 3 false positives. The assessment stage is conservative — it rarely claims injectability where none exists, and instead misses opportunities.

RQ2 — survival. Nearly 1,000 raw generations reduce to 193 non-duplicate contracts, of which **32 survive all four steps: a 16.58 % survival rate**, spanning 25 vulnerability types. The per-step decomposition is the finding:

Step	Survivors	Note
Raw generations	~997	
After deduplication	193	an 80 % reduction
A — compiles and executes	150	77.72 % pass; 26 of 43 failures are the injector emitting constructs unsupported by pre-0.4.24 compilers
B — injection verified	89	constraint-compliant injection is hard even when Phase II predicted injectability

Step	Survivors	Note
C — business logic preserved	44	the bottleneck: the largest relative drop in the pipeline
D — vulnerability confirmed	32	final ground truth

Step C is where the paper earns its keep. Three recurring failure patterns are named: access control removed or weakened (transfers permitted without sufficient funds), fund-transfer logic altered (storage updates unrelated to the injected vulnerability), and outright replacement of function semantics (a `Kill` function that no longer terminates the contract). The lesson generalises past Solidity: **making a program vulnerable is easy; making it vulnerable *and* otherwise unchanged is the hard part**, and it is not detectable by compilation.

Two structural regularities explain the losses. Simpler targets survive more often — the low-LOC/low-cyclomatic-complexity class has the highest survival, the high/high class the strongest reduction, with the effect concentrated in Step C. And localised defect types survive better: grouped by ODC class, `Checking` dominates with 11 of the 25 surviving types, followed by Timing/Serialization (5) and Algorithm/Method (4), while structural classes such as Algorithm/Method and Assignment/Initialization show much lower survival because they need changes that are hard to reconcile with the injection constraints.

The authors publish their own bias, which is the reason the dataset is usable at all: the validated ground truth is skewed toward checking-related vulnerabilities, taxonomy coverage narrows substantially as contracts move through the pipeline, and consumers are told in as many words to account for this distribution before drawing conclusions about tool effectiveness.

RQ3 — using the dataset. Three static analysers on the validated contracts:

Tool	TP	FP	FN	Analysis failures	Precision	Recall
Remix	11	3	5	13	0.786	0.688
Slither	11	0	8	13	1.000	0.579
Solhint	12	2	5	13	0.857	0.706

The authors attach their own caveat and it should travel with the numbers: the evaluation covers 19 contracts, so one TP or FP shifts the metrics substantially — treat these as indicative, not conclusive. The structural conclusion is the durable one: coverage is partially complementary, each tool catches something the others miss, no tool has both perfect precision and high coverage, and combining analysers is necessary. Only 9 of the 25 types get at least one confirmed detection, with true positives concentrating in the types with the clearest syntactic signature.

The cleanest finding in the paper is incidental. 13 of 32 validated contracts (40.6 %) could not be analysed by any of the three tools. All 13 originate from four targets on legacy Solidity (0.3.x, 0.4.6, 0.4.11) using deprecated constructs — `sha3`, legacy `delegatecall`, `callcode`, unchecked `send`. Crucially, running the same tools on the *unmodified originals* produces the same errors, which attributes the failure to the analysers' handling of legacy code rather than to anything the injection did.

Two honest negatives

Convergence. The 80 % reduction from ~997 raw generations to 193 distinct contracts means the model, under fixed constraints, repeatedly produces near-identical outputs. Increasing the number of generation runs yields diminishing returns. The authors did not test temperature variation, prompt perturbation or multi-model ensembles, and flag those as the obvious next step. They also note their textual deduplication has error in both directions — formatting-only variants counted as distinct, textually identical outputs on different execution paths merged — and suggest symbolic-execution-based semantic equivalence as a more robust criterion.

Model size is not the lever. A long-context proprietary model (Claude Sonnet 5, 1M-token window) run as injector on three contracts passed 9 of 25 candidates (36 %), concentrating in the same localised types the small model favoured; value-transfer, transaction-order, reentrancy and arithmetic variants were rejected again. The authors' conclusion — that producing a correctly typed, exploitable defect while preserving business logic is intrinsic to the task rather than a function of model capacity — is appropriately hedged as a three-contract probe with no deduplication.

Why it is worth the time

The value is not the 32 contracts; it is the pipeline shape and the diagnosis. Four sequential validators with published per-step attrition is a template that transfers directly to injection work in any language, and the identification of business-logic preservation as the dominant bottleneck — with a concrete proposal to automate it via differential testing against the original — is actionable. The 40.6 % legacy-code analysis failure is a separate, well-attributed result about the state of Solidity tooling. And the self-published selection bias is what separates a dataset you can use carefully from one you cannot use at all.

1.4 · CODE REPRESENTATION · Behaviour trees fit 260/260 longer Java samples in a 4096-token window where ASTs fit 86 — a representation-budget result wearing a detection paper's clothes 🙌 😊 📄

Towards Behavior Tree–Guided Vulnerability Detection with Lightweight LLMs

Authors: Enna Basic, Alberto Giarretta (Department of Computer Science, Örebro University, Sweden).

Venue: arXiv:2609.01758v1, cs.CR, 1 Sep 2026. ACM formatting with CCS concepts, but no conference or journal is named in the paper. The authors describe the work as a preliminary investigation.

Links. [arXiv:2609.01758](https://arxiv.org/abs/2609.01758). **Licence: none stated** — no Creative Commons or ACM copyright notice appears in the PDF, so nothing is mirrored here and no figures are reproduced. **Evidence base: full text retrieved and read. Every number below is transcribed from the paper.**

The gap being targeted

How you present code to a model is a design variable, and the structured-representation literature has mostly converged on ASTs and their enrichments — SCALE augments AST nodes with auto-generated comments, VulTrLM decomposes ASTs into commented subtrees. All of these buy structure at the cost of tokens, and for a locally deployed quantised model with a small context window, that cost is not incidental: it decides whether the input is processable at all.

The paper's question is whether a more compact structured representation exists. Behaviour trees — Sequence, Selector, Condition and Action nodes, borrowed from robotics and game AI — encode

control flow, conditions and executable actions without the syntactic completeness that makes ASTs verbose. To the authors' knowledge they have not previously been used as an input representation for LLM-based vulnerability detection.

Setup

Java source is parsed to an AST and then converted to a BT by explicit mapping rules. Two balanced datasets are built from the NIST Juliet Java suite, each with equal vulnerable and non-vulnerable samples per CWE: a **short-sample set** (10 CWEs × 20 cleaned samples = 200, sized so all three representations fit a 4096-token window) and a **longer-sample set** (13 CWEs × 20 = 260, sized so source and BT fit while many ASTs do not). 460 samples in total.

One model throughout: **Mistral Small 3.2 24B, Q4_K_M**, quantised and locally run, context 4096 tokens, max 1024 generated, repetition penalty 1.15, repeated across 10 runs which produced identical results. The prompt is four lines — analyse the following `<type>` for vulnerabilities, return the CWE if possible. Outputs are scored three ways: exact CWE match, keyword match against the official CWE name, and embedding similarity to the official CWE description.

Results

Short samples (all three representations fit). The comparison is close to a wash:

Representation	Precision	Recall	F1	Accuracy
<i>Keyword matching</i>				
Source code	0.60	0.75	0.67	0.63
AST	0.55	0.72	0.62	0.56
BT	0.57	0.83	0.67	0.60
<i>Exact CWE matching</i>				
Source code	0.59	0.54	0.57	0.59
AST	0.55	0.59	0.57	0.55
BT	0.56	0.68	0.62	0.58

BT takes recall, source code takes precision and accuracy — a coverage-versus-precision trade-off, with BT producing more false positives. AST shows no consistent advantage over raw source at all, which is itself worth noting: syntactic detail does not automatically help a lightweight model. Mean semantic similarity is effectively indistinguishable across the three (0.4660 source, 0.4652 AST, 0.4560 BT), with standard deviations around 0.13 — larger than the gaps.

Longer samples (AST excluded, it does not fit). BT improves on source across the board: F1 0.59 → 0.64 under keyword matching and 0.56 → 0.58 under exact CWE, driven by recall (0.73 → 0.83 and 0.63 → 0.69). BT takes the highest F1 in 8 of 13 CWE categories and the higher mean semantic similarity in 8 of 13 (0.4470 vs 0.4411 overall — again inside the noise).

The result that does not depend on any of that. Input coverage by context window on the longer-sample set:

Representation	4096	8192	16384
Source code	260/260	260/260	260/260
AST	86/260	156/260	222/260
BT	260/260	260/260	260/260

Quadrupling the context window still leaves AST short of full coverage, while BT is complete at the smallest budget tested. And enlarging the window does not buy detection quality: BT's overall F1 is 0.58 at 4096, 0.54 at 8192 and 0.58 at 16 384; source code is 0.56, 0.58, 0.56. More context improves *feasibility*, not accuracy.

How far it travels, and what would have to change

The scope is narrow and the authors say so. One quantised model, one synthetic suite, 460 samples, no baseline beyond the two alternative representations, and Juliet's controlled examples are not real-world code — synthetic vulnerabilities are cleaner and more localised than anything in a production repository, which plausibly flatters a control-flow-oriented representation. The evaluation metrics carry their own limitations: keyword matching depends on the official CWE name, semantic similarity on the choice of embedding model, and a response can be security-correct while scoring as wrong under exact CWE matching.

The detection numbers should therefore be read as suggestive at best — the gaps are within the spread of the per-CWE variation. What holds up independently is the token-budget finding, which is a direct property of the representations and reproducible from the conversion rules alone. Confirming the rest would take several models across the quantisation range, at least one real-world dataset such as a CVE-derived corpus, and a fixed-token-budget comparison that controls for input length rather than letting representations differ in size.

Why it is worth the time

Chiefly as a budget argument. For anyone running local models on constrained hardware — the setting the paper explicitly targets — “the structured representation you wanted does not fit” is a real and under-discussed constraint, and BTs are a plausible answer with a documented AST-to-BT mapping. The observation that ASTs offer no consistent advantage over raw source for a lightweight model is a useful negative result on its own. Treat the F1 tables as a pilot, and the coverage table as the finding.

1.5 · FUZZING · Control-flow restructuring plus freshness-decayed seed scheduling for pre-silicon processor verification — listed as a pointer, full text not retrievable 🙌 😊 🗑️

ConFluxFuzz: Control-Flow Restructuring and Freshness-Decayed Scheduling for Processor Fuzzing

Authors: R. Fang, H. Wang, X. Chen, W. Cui, N. Cui, L. Chen, G. Shi, et al. (author list as printed by the alert; not verified against the paper).

Venue: IEEE Transactions, 2026.

Links. [IEEE Xplore, document 11674189](#).

Evidence base: alert snippet only — no deep read was possible. The publisher page is behind a paywall and renders client-side, the paper is not present in any open index or preprint server that could

be reached, and the alert snippet itself is truncated mid-sentence. Rather than reconstruct a summary from the title, the entry is left as a pointer.

What can be said from the snippet

The stated problem is that modern processors carry increasingly complex control logic, which makes efficient and scalable state-space exploration a long-standing challenge in pre-silicon verification, and that recent coverage-guided processor fuzzing techniques do not fully address it. The two named mechanisms — control-flow restructuring, and a seed-scheduling policy that decays a seed's priority as its coverage contribution ages — are both, in principle, transferable to software fuzzing: freshness decay is a general answer to the problem of a seed pool that keeps re-selecting inputs whose marginal coverage has already been harvested.

That is the extent of what the available evidence supports. Readers with IEEE access may find the scheduling policy worth extracting; no claim is made here about its reported results.

Cross-Paper Synthesis

Four of the five papers are arguments about when to stop spending, and they land on the same structural answer from different directions. EarlyEval stops a benchmark run once the outcome is statistically determined. The behaviour-tree study stops paying tokens for syntactic completeness the model does not use. The injection paper stops hand-curating ground truth — and discovers where the automation actually breaks. The position paper stops treating artefact-level validation as evidence once an agent authors both the artefact and its test. In each case the move is the same: identify the point past which additional work stops producing additional evidence, and characterise that point precisely enough to act on it.

Two of them independently locate the hard part in semantics rather than syntax. The injection pipeline's dominant loss is Step C, business-logic verification — 89 survivors down to 44 — because compilation and even correct injection say nothing about whether the contract still does what it did. The position paper reaches the same wall from the top down: an agent that writes the implementation and the tests can be internally consistent and externally wrong, and a verifier agent can co-adapt until the tests ratify the bugs. Both conclude that the missing artefact is an independent statement of intended behaviour, and both propose the same family of fix — differential comparison against a reference version, or an oracle inferred from something other than the implementation itself. That is also, notably, what yesterday's PatchGuru was doing for patches.

All four full reads disclose the shape of their own failure, and it is what makes them usable.

EarlyEval reports that its success predictor collapses to zero coverage on one of three benchmarks and explains the mechanism. The injection paper publishes its ground truth's selection bias and tells consumers not to over-read it. The behaviour-tree paper labels itself preliminary and lists the threats. The position paper devotes a section to the two views that would falsify it. The correlation is not accidental: a result stated with its boundary attached is one you can reuse outside the paper's own setting, which is exactly what a paper that hides the boundary cannot offer.

A dissent worth registering. The one place the set is weakest is where its evidence is thinnest: two of the five report headline differences smaller than their own measurement spread — the behaviour-tree semantic-similarity gaps sit well inside a 0.13 standard deviation, and the 19-contract analyser

comparison moves materially on a single classification. Both sets of authors say so. The reader's job is to carry the caveat forward rather than the number, and this report has tried to quote them together.

Writing & Rationale Insights

Lead with the cost table. EarlyEval opens on measured dollar figures from a public leaderboard — \$715 for one SWE-bench pass, \$2,270 for SWE-bench Multimodal — before proposing anything. The motivation is then not an assertion about expense but a citation, and every later percentage inherits a unit the reader already believes in.

Put the strongest evidence where the paper is weakest. EarlyEval's success predictor works on one of its three benchmarks. The paper reports the collapse in the main results table, quantifies it across every threshold, and supplies a mechanism — scaffolds dictate the structural rhythm the behavioural features encode, so an unseen scaffold perturbs them while an unseen model does not. A weakness explained is a contribution; a weakness omitted is a liability waiting for a reviewer.

Publish the attrition, not just the yield. “16.58 % survival” is an unpromising headline. “89 to 44 at business-logic verification” is a finding, because it tells the next person which stage to attack. Reporting per-step attrition converts a disappointing number into a research direction — and it is what lets a reader decide whether the loss would apply to their own setting.

Disclose your dataset's bias in the abstract's voice, not a threats section. The injection paper states inside its own RQ2 answer that the ground truth skews to checking-class defects and that consumers should account for the distribution. Placing that where the result is claimed, rather than four pages later, is the difference between a caveat a reader acts on and one they skim.

Pre-register your falsifiers. The position paper names the two views that would defeat it — durable partnership, and scaling dissolves the problems — gives each a fair statement, and then answers. This is both more persuasive than a one-sided case and more useful, because it tells the reader what evidence would change the picture. Position papers that skip this step read as advocacy regardless of how careful the underlying argument is.

Let the sturdy result lead, even if it is not the one you set out to find. The behaviour-tree study's F1 differences are inside its own noise; its coverage table (86/260 versus 260/260 at 4096 tokens) is a hard property of the representations. A paper reads much more strongly when the claim it foregrounds is the claim its evidence best supports — and a reader who reframes it that way gets more out of it than one who takes the abstract's ordering at face value.

Say what you could not retrieve. The fifth entry above is a pointer rather than a summary because the full text was not reachable. Marking the boundary of the evidence costs one sentence and preserves the meaning of every other card in the report.